REVISION CHEAT SHEET

REPOSITORY / UOW / MAPPING

Repository, Unit of Work, Service Layer, and Mapping Patterns

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. Why These Patterns Are Grouped Together
2. Running Fundraising Example
PART 1 — REPOSITORY PATTERN
3. What Is the Repository Pattern?
4. Repository Does Not Mean “One Class Per Table”
5. Repository as an Aggregate Boundary
6. Repository Interface
7. Repository Implementation
8. Should Repository Interfaces Be in Domain or Application?
9. DbContext Already Looks Like a Repository
10. Direct DbContext Design
11. When a Custom Repository Adds Value
12. When a Repository Adds Little Value
13. Generic Repository — Advantages
14. Generic Repository — Limitations
15. Should a Repository Return IQueryable<T> ?
16. IQueryable Leak Example
17. Better Command/Query Separation
18. Query Service Example
19. Why Projection Matters
20. Tracking vs No-Tracking in Repositories
21. Read-Only Queries
22. Tracking Is Not Always Slower
23. Repository Loading Strategy
PART 2 — UNIT OF WORK
24. What Is Unit of Work?
25. DbContext Already Is a Unit of Work
26. Change Tracker
27. Unit of Work Scope
28. Unit of Work Interface
29. Is IUnitOfWork Necessary?
30. Repository Should Not Usually Save Independently
31. Better Unit-of-Work Boundary
32. SaveChanges Transaction Behavior
33. When an Explicit Transaction Is Needed
34. Keep Transactions Short
35. Database Transaction Does Not Cover External Systems
36. Optimistic Concurrency
37. SQL Server rowversion Example
PART 3 — SERVICE LAYER
38. What Is the Service Layer Pattern?
39. Service Layer Is Not the Same as “Services Folder”
40. Application Service Example
41. What Belongs in the Service Layer?
42. What Does Not Belong There?
43. Application Service vs Domain Service
Application Service
Domain Service
44. Application Service vs Infrastructure Service
45. Service Layer and Controllers
46. Service Layer and Vertical Slices
47. Transaction Boundary Usually Belongs to the Use Case
Diagram 1 — Responsibility Separation
PART 4 — MAPPING PATTERNS
48. Why Mapping Exists
49. Why Not Use One Model Everywhere?
50. DTO — Data Transfer Object
51. Request DTO Is Not Necessarily a Domain Entity

52. Mapping Boundary
53. Manual Mapping
54. Manual Mapping Is Often Best
55. Mapping Extension Methods
56. Dedicated Mapper Class
57. Mapper Should Not Become Service Layer
58. AutoMapper
59. AutoMapper Benefits
60. AutoMapper Costs
61. AutoMapper Map vs ProjectTo
- Map
- ProjectTo

62. ProjectTo Must Respect Query Provider Rules
63. ProjectTo Has Limitations
64. Explicit Select vs ProjectTo
65. Mapping Domain to DTO
66. DTO to Domain Mapping Needs More Care
67. Why ReverseMap Can Be Dangerous
68. Persistence Model vs Domain Model
- Same Entity Model
- Separate Models

69. EF Core Can Map Rich Domain Models
70. Integration Contract Mapping
Diagram 2 — Mapping Boundaries
PART 5 — PUTTING THE PATTERNS TOGETHER
71. Create Gift Use Case
72. Application Service
73. Pattern Responsibilities
74. Complete Gift + Outbox
75. Read-Side Query
Diagram 3 — Command vs Query Architecture
PART 6 — TESTING
76. Repository Testing
77. Do Not Mock EF Core Internals Excessively
78. Application Service Unit Test
79. Repository Integration Test
PART 7 — PERFORMANCE
80. Avoid Repository N+1 Queries
81. Project Only What You Need
82. Avoid Premature ToList
83. Repository Method Should Not Materialize Too Early
84. Change Tracker Cost
85. SaveChanges Batching
PART 8 — SECURITY
86. Repository Is Designed as One
87. Tenant Filtering
88. Mapping Can Leak Sensitive Fields
89. AutoMapper Security Warning
90. Repository Does Not Replace Authorization
PART 9 — MEMORY AND THREAD SAFETY
91. Repository Lifetime
92. DbContext Is Not Thread-Safe
93. Long-Lived Unit of Work Is Dangerous
PART 10 — COMMON MISTAKES
94. Repository per Table
95. Repository Calls SaveChanges
96. Service Layer Contains Domain Logic
97. Domain Entity Contains Repository
98. Repository Returns DTOs for Everything
99. Mapping Business Rules
100. Blind Reverse Mapping
101. Generic Repository Hiding EF Features
102. Mocking Repository Without Integration Tests
103. Mapping After Loading Huge Graph
PART 11 — BEST PRACTICES
104. Repository Best Practices
105. Unit of Work Best Practices
106. Service Layer Best Practices
107. Mapping Best Practices
PART 12 — PRACTICAL EXERCISES
- Exercise 1 — Repository Boundary
- Exercise 2 — Remove SaveChanges from Repository
- Exercise 3 — Add Outbox Transaction
- Exercise 4 — Read Projection
- Exercise 5 — Service Layer
- Exercise 6 — Domain Rule
- Exercise 7 — Manual Mapping
- Exercise 8 — AutoMapper Projection
- Exercise 9 — Multi-Tenant Repository
- Exercise 10 — Concurrency
- Exercise 11 — Repository Integration Test
- Exercise 12 — Architecture Review

PART 13 — EXACTLY 25 INTERVIEW QUESTIONS
1. What is the Repository pattern?
2. Does EF Core require the Repository pattern?

REPOSITORY / UOW / MAPPING

3. When should you create a custom Repository over DbContext?
4. Should there be one Repository per database table?
5. What are the advantages and disadvantages of a generic Repository?
6. Should a Repository return IQueryable<T> ?
7. What is the Unit of Work pattern?
8. How does DbContext implement Unit of Work?
9. Do you need an IUnitOfWork abstraction on top of DbContext?
10. Should Repository methods call SaveChanges internally?
11. Is SaveChanges transactional?
12. When do you need an explicit EF Core transaction?
13. What is the Service Layer pattern?
14. What is the difference between Application Service and Domain Service?
15. What is the difference between Service Layer and a Repository?
16. Why should business rules not live primarily in Application Service?
17. What is a DTO?
18. Why should API DTOs be separate from domain entities?
19. What are the benefits of manual mapping?
20. When is AutoMapper useful?
21. What is the difference between AutoMapper Map and ProjectTo?
22. Why can direct Select be better than mapping entities after loading?
23. Why is blindly mapping DTOs back into domain entities dangerous?
24. How should command and query persistence differ in a CQRS-style application?
25. How would you use Repository, Unit of Work, Service Layer, and Mapping in the Fundraising Platform?

PART 14 — QUICK REVISION

PART 15 — FINAL DECISION FLOW

PART 16 — FINAL END-TO-END FUNDRAISING FLOW

REPOSITORY / UOW / MAPPING

1. Why These Patterns Are Grouped Together

In a typical ASP.NET Core backend, a request often follows a flow like:

EXAMPLE / FLOW

```
HTTP Request
    ↓
Controller / Endpoint
    ↓
Application Service
    ↓
Domain Model
    ↓
Repository
    ↓
Unit of Work
    ↓
EF Core DbContext
    ↓
SQL Server
```

At the same time, data may need to move between several different representations:

EXAMPLE / FLOW

```
HTTP Request DTO
    ↓
Application Command
    ↓
Domain Entity
    ↓
Persistence
```

and back:

EXAMPLE / FLOW

```
Database / Entity
    ↓
Query Projection
    ↓
Application DTO
    ↓
HTTP Response
```

That means four architectural concerns frequently meet in one use case:

Concern	Main responsibility
Repository	Encapsulates persistence access to domain/application objects
Unit of Work	Coordinates a persistence transaction/commit boundary
Service Layer	Coordinates an application use case
Mapping	Translates data between representations/boundaries

These patterns are useful only when they make boundaries clearer.

They become harmful when they simply add:

EXAMPLE / FLOW

```
Controller
    ↓
Service
    ↓
Manager
    ↓
Repository
    ↓
Generic Repository
    ↓
DbContext
```

without each layer owning a meaningful responsibility.

2. Running Fundraising Example

Throughout this topic, assume the core Fundraising relationship:

EXAMPLE / FLOW

```
Constituent
    ↓
Gift
    ↓
GiftAllocation
    ↓
Fund

Gift
    ↓
Campaign
```

The important tables/entities include:

EXAMPLE / FLOW

```
constituent.Constituent
```

```
fundraising.Gift
fundraising.GiftAllocation
fundraising.Fund
fundraising.Campaign
integration.OutboxMessage
```

REPOSITORY / UOW / MAPPING

A typical use case is:

> Complete a Gift after verifying its allocations, persist the status change, and create an OutboxMessage in the same database transaction.

That one use case lets us see all four patterns working together.

PART 1 — REPOSITORY PATTERN

3. What Is the Repository Pattern?

A Repository represents a collection-like persistence boundary for domain/application code.

Instead of business code saying:

C#

```
await dbContext.Gifts
    .Include(g => g.Allocations)
    .ThenInclude(a => a.Fund)
    .SingleAsync(
        g =>
            g.TenantId == tenantId
            &&
            g.GiftId == giftId,
        cancellationToken);
```

the business/application layer can say:

C#

```
Gift? gift =
    await giftRepository.FindForCompletionAsync(
        tenantId,
        giftId,
        cancellationToken);
```

The caller expresses:

EXAMPLE / FLOW

```
What it needs
```

rather than:

EXAMPLE / FLOW

```
How EF Core loads it
```

Microsoft's current .NET architecture guidance describes Repository as an intermediary between the domain model and data mapping/persistence. It also notes that EF Core's `DbContext` already implements Repository and Unit-of-Work concepts internally.

4. Repository Does Not Mean “One Class Per Table”

This is one of the most important rules.

A poor design often creates:

```
EXAMPLE / FLOW
GiftRepository
GiftAllocationRepository
CampaignRepository
FundRepository
AddressRepository
ConstituentAddressRepository
```

because:

```
EXAMPLE / FLOW
one SQL table
=
one repository
```

That is not the DDD meaning of Repository.

When using aggregate-oriented domain modeling, Microsoft recommends repositories around **aggregate roots**, not around every table. The aggregate root controls transactional consistency for the objects inside its aggregate.

For example:

```
EXAMPLE / FLOW
Gift Aggregate
    ↓
Gift
    └─ GiftAllocation
```

may naturally have:

```
EXAMPLE / FLOW
IGiftRepository
```

rather than:

```
EXAMPLE / FLOW
IGiftRepository
+
IGiftAllocationRepository
```

if `GiftAllocation` should only be changed through its `Gift`.

5. Repository as an Aggregate Boundary

Suppose the domain says:

```
EXAMPLE / FLOW
Gift
    ↓
owns GiftAllocation lifecycle
```

Then application code should ideally not do this:

```C#
await giftAllocationRepository.DeleteAsync(
    allocationId,
    cancellationToken);
```

because that allows callers to bypass Gift's invariants.

Instead:

```C#
Gift gift =
    await giftRepository.FindAsync(
        tenantId,
        giftId,
        cancellationToken)
    ?? throw new GiftNotFoundException(giftId);

gift.RemoveAllocation(
    allocationId);

await unitOfWork.SaveChangesAsync(
    cancellationToken);
```

Now:

```EXAMPLE / FLOW
Repository loads aggregate
    ↓
Aggregate enforces rules
    ↓
Unit of Work persists result
```

6. Repository Interface

A practical Gift repository might look like:

```C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);

    Task<Gift?> FindForCompletionAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);

    Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Notice what is **not** necessarily present:

```C#
Update(Gift gift);
Delete(Gift gift);
```

```
GetAll();
IQueryable<Gift> Query();
```


A repository should expose operations that make sense for the domain/application boundary.

7. Repository Implementation

C#

```
public sealed class GiftRepository :
    IGiftRepository
{
    private readonly FundraisingDbContext _dbContext;

    public GiftRepository(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
    {
        return _dbContext.Gifts
            .SingleOrDefaultAsync(
                gift =>
                    gift.TenantId == tenantId
                    &&
                    gift.GiftId == giftId,
                cancellationToken);
    }

    public Task<Gift?> FindForCompletionAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
    {
        return _dbContext.Gifts
            .Include(gift => gift.Allocations)
            .SingleOrDefaultAsync(
                gift =>
                    gift.TenantId == tenantId
                    &&
                    gift.GiftId == giftId,
                cancellationToken);
    }

    public async Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        await _dbContext.Gifts.AddAsync(
            gift,
            cancellationToken);
    }
}
```

EF Core details remain inside:

EXAMPLE / FLOW

```
Infrastructure
```

while Application depends on:

EXAMPLE / FLOW

```
IGiftRepository
```

REPOSITORY / UOW / MAPPING

8. Should Repository Interfaces Be in Domain or Application?

There is no universal rule that every project must follow.

A DDD-heavy design may place aggregate repository contracts in:

EXAMPLE / FLOW

```
Domain
```

because the repository concept belongs to the aggregate's persistence abstraction.

A Clean Architecture design may place ports needed by use cases in:

EXAMPLE / FLOW

```
Application
```

because Application defines the capabilities it requires from Infrastructure.

The important dependency rule is:

EXAMPLE / FLOW

```
Infrastructure implementation
    ↓
depends on core abstraction
```

rather than:

EXAMPLE / FLOW

```
Domain/Application
    ↓
depends directly on EF repository implementation
```

Microsoft's DDD architecture guidance places repository abstractions in the domain/core area and implementations in Infrastructure.

9. DbContext Already Looks Like a Repository

Consider:

C#

```
dbContext.Gifts
```

which is:

C#

```
DbSet<Gift>
```

It already supports operations such as:

C#

```
Add(...)
FindAsync(...)
```

```
Where(...)
SingleAsync(...)
Remove(...)
```

REPOSITORY / UOW / MAPPING

Then:

```C#
dbContext.SaveChangesAsync()
```

coordinates persistence.

This is why Microsoft explicitly states that EF Core's `DbContext` embodies both Repository and Unit-of-Work concepts.

Therefore:

> **Custom Repository is optional when using EF Core.**

This is a major architectural decision, not a required ceremony.

10. Direct DbContext Design

A simple application service could directly use:

```C#
public sealed class CreateGiftService
{
    private readonly FundraisingDbContext _dbContext;

    public CreateGiftService(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task<long> CreateAsync(
        CreateGiftCommand command,
        CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                command.TenantId,
                command.ConstituentId,
                command.TotalAmount);

        _dbContext.Gifts.Add(
            gift);

        await _dbContext.SaveChangesAsync(
            cancellationToken);

        return gift.GiftId;
    }
}
```

This can be perfectly reasonable for:

EXAMPLE / FLOW

```
Simple application

Simple CRUD
```

```
Limited domain complexity

No need to isolate persistence vocabulary
```

REPOSITORY / UOW / MAPPING

Microsoft's guidance explicitly says using `DbContext` directly can be the simplest choice and custom repositories are not mandatory.

11. When a Custom Repository Adds Value

A repository is more useful when it creates a meaningful boundary.

For example:

C#
```
Task<Gift?> FindForCompletionAsync(...);
```

expresses a domain/use-case concept.

It hides:

EXAMPLE / FLOW
```
Include allocations
Tracking behavior
Query filters
EF Core implementation
Persistence provider
```

Another useful example:

C#
```
Task<bool> ExternalReferenceExistsAsync(
    long tenantId,
    string externalReference,
    CancellationToken cancellationToken);
```

This represents an application capability rather than simply exposing EF Core.

12. When a Repository Adds Little Value

Consider:

C#
```
public interface IRepository<T>
{
    IQueryable<T> GetAll();

    Task<T?> GetByIdAsync(
        long id);

    void Add(
        T entity);

    void Update(
        T entity);

    void Delete(
        T entity);
}
```

implemented as:

```C#
public sealed class Repository<T>
{
    private readonly DbSet<T> _dbSet;

    public IQueryable<T> GetAll()
        =>
        _dbSet;

    public void Add(
        T entity)
        =>
        _dbSet.Add(
            entity);
}
```

This often becomes:

```EXAMPLE / FLOW
DbSet<T>
    ↓
renamed
```

rather than a useful abstraction.

You now maintain another API without gaining meaningful domain isolation.

13. Generic Repository — Advantages

A generic repository can reduce repeated infrastructure code:

```C#
public interface IRepository<TEntity>
    where TEntity : class
{
    Task<TEntity?> FindAsync(
        long id,
        CancellationToken cancellationToken);

    Task AddAsync(
        TEntity entity,
        CancellationToken cancellationToken);
}
```

Potential benefits:

```EXAMPLE / FLOW
Less repetitive CRUD code

Shared common behavior

Centralized persistence conventions
```

14. Generic Repository — Limitations

The problem is that different aggregates have different needs.

Gift needs:

EXAMPLE / FLOW

```
Load allocations
Check external reference
Load Campaign relationship
Tenant filtering
```

REPOSITORY / UOW / MAPPING

Campaign may need:

EXAMPLE / FLOW

```
Load active funds
Campaign performance queries
```

A lowest-common-denominator generic repository may force awkward code.

You eventually end up with:

EXAMPLE / FLOW

```
GenericRepository
    +
GiftRepository
    +
CampaignRepository
```

where the generic abstraction contributes little.

A better rule is:

> ⓘ Use generics only for behavior that is genuinely generic.

15. Should a Repository Return IQueryable<T>?

Example:

C#

```
IQueryable<Gift> Query();
```

Advantage:

EXAMPLE / FLOW

```
Caller can compose efficient SQL.
```

But the cost is important.

The caller now understands:

EXAMPLE / FLOW

```
LINQ provider behavior
EF Core translation
Tracking
Includes
Database execution timing
```

That means persistence semantics leak through the repository boundary.

16. IQueryable Leak Example

```C#
IQueryable<Gift> query = giftRepository.Query();

List<GiftResponse> gifts =
    await query
        .Include(gift => gift.Allocations)
        .AsNoTracking()
        .Select(...)
        .ToListAsync();
```

At this point the Application layer already knows:

EXAMPLE / FLOW

```
EF Core-style query composition
```

So ask:

> ⓘ What is the repository actually hiding?

Returning `IQueryable<T>` can be reasonable in some pragmatic architectures, but it weakens persistence abstraction.

17. Better Command/Query Separation

A strong approach is:

EXAMPLE / FLOW

```
Commands
    ↓
Repository + Domain Aggregate

Queries
    ↓
Direct optimized projection
```

Microsoft's DDD persistence guidance explicitly allows query paths outside repositories when using CQRS, while repositories remain focused on aggregate updates and consistency boundaries.

For example:

EXAMPLE / FLOW

```
CompleteGiftCommand
    ↓
IGiftRepository
    ↓
Gift aggregate
```

while:

EXAMPLE / FLOW

```
GetGiftSummaryQuery
    ↓
DbContext projection
    ↓
GiftSummaryDto
```

This is often much cleaner than forcing every read query through an aggregate repository.

18. Query Service Example

C#

```
public sealed class GiftQueries
{
    private readonly FundraisingDbContext _dbContext;

    public GiftQueries(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public Task<GiftSummaryDto?>
        GetByIdAsync(
            long tenantId,
            long giftId,
            CancellationToken cancellationToken)
    {
        return _dbContext.Gifts
            .AsNoTracking()
            .Where(
                gift =>
                    gift.TenantId == tenantId
                    &&
                    gift.GiftId == giftId)
            .Select(
                gift =>
                    new GiftSummaryDto(
                        gift.GiftId,
                        gift.TotalAmount,
                        gift.Status,
                        gift.Campaign != null
                            ? gift.Campaign.Name
                            : null))
            .SingleOrDefaultAsync(
                cancellationToken);
    }
}
```

No domain aggregate is needed because the operation is:

EXAMPLE / FLOW

```
read only
```

and no business state is being changed.

19. Why Projection Matters

Compare:

C#

```
Gift gift =
    await dbContext.Gifts
        .Include(g => g.Campaign)
        .SingleAsync(...);
```

then:

```C#
return new GiftSummaryDto(
    gift.GiftId,
    gift.TotalAmount,
    gift.Campaign!.Name);
```

with direct projection:

```C#
return await dbContext.Gifts
    .Where(...)
    .Select(
        gift =>
            new GiftSummaryDto(
                gift.GiftId,
                gift.TotalAmount,
                gift.Campaign!.Name))
    .SingleAsync(
        cancellationToken);
```

EF Core documentation recommends projecting only the properties required because retrieving complete entities transfers unnecessary columns and creates unnecessary materialization work.

20. Tracking vs No-Tracking in Repositories

When loading an aggregate for update:

```C#
Gift? gift =
    await dbContext.Gifts
        .Include(gift => gift.Allocations)
        .SingleOrDefaultAsync(...);
```

tracking is useful because:

EXAMPLE / FLOW

```
modify Gift
    ↓
SaveChanges
    ↓
EF detects modifications
```

EF Core tracks entity changes and uses those tracked entities when `SaveChanges` is called.

21. Read-Only Queries

For a read-only projection:

```C#
await dbContext.Gifts
    .AsNoTracking()
    .Where(...)
    .Select(...)
    .ToListAsync();
```

no tracking is usually appropriate.

Microsoft describes no-tracking queries as useful for read-only scenarios because EF does not need to set up normal change-tracking information.

22. Tracking Is Not Always Slower

Do not memorize:

```
EXAMPLE / FLOW
AsNoTracking
=
always faster
```

Tracking provides:

```
EXAMPLE / FLOW
identity resolution
```

which can avoid creating duplicate instances of the same entity in some query results.

EF Core's tracking documentation specifically notes that tracking queries perform identity resolution, while ordinary no-tracking queries do not.

The correct question is:

```
EXAMPLE / FLOW
Do I need tracked entities for this operation?
```

23. Repository Loading Strategy

A repository method should load what the aggregate operation actually needs.

For example:

```
C#
FindForCompletionAsync()
```

might need:

```
EXAMPLE / FLOW
Gift
GiftAllocations
Fund state
```

but:

```
C#
FindForReferenceChangeAsync()
```

might need much less.

Avoid a repository method that always does:

```
C#
.Include(g => g.Allocations)
.Include(g => g.Campaign)
.Include(g => g.Constituent)
.Include(g => g.Receipts)
.Include(...)
```

for every use case.

EF Core warns that eager-loading collection navigations can produce expensive queries in some situations.

PART 2 — UNIT OF WORK

24. What Is Unit of Work?

Unit of Work coordinates a set of related persistence changes as one logical operation.

For example:

EXAMPLE / FLOW

```
Complete Gift
    ↓
Change Gift.Status
    ↓
Add OutboxMessage
    ↓
Update Campaign statistic
    ↓
Commit together
```

The desired result is:

EXAMPLE / FLOW

```
All succeed
```

or:

EXAMPLE / FLOW

```
all fail
```

rather than partial state.

25. DbContext Already Is a Unit of Work

An EF Core `DbContext`:

EXAMPLE / FLOW

```
Tracks entities
Tracks changes
Coordinates inserts/updates/deletes
Calls SaveChanges
Participates in transactions
```

Microsoft explicitly describes `DbContext` as implementing Unit of Work and recommends treating it as a short-lived unit-of-work object.

26. Change Tracker

Suppose:

C#

```
Gift gift =
    await dbContext.Gifts
        .SingleAsync(...);

gift.Complete();
```

You haven't issued an SQL `UPDATE` yet.

EF's change tracker understands:

EXAMPLE / FLOW

```
Gift
Original Status = Pending
Current Status = Completed
```

Then:

C#

```
await dbContext.SaveChangesAsync(
    cancellationToken);
```

causes EF to generate the required database changes.

EF Core states that tracked entity changes drive the changes persisted by `SaveChanges`.

27. Unit of Work Scope

A Unit of Work should normally be short-lived:

EXAMPLE / FLOW

```
Begin use case
    ↓
Load required entities
    ↓
Modify entities
    ↓
Save
    ↓
Dispose context
```

Do not keep one DbContext for:

EXAMPLE / FLOW

```
Entire application lifetime
```

or:

EXAMPLE / FLOW

```
hours of unrelated requests
```

EF Core explicitly describes `DbContext` as designed for a short-lived unit of work.

28. Unit of Work Interface

Some architectures create:

C#

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(
        CancellationToken cancellationToken);
}
```

Then:

C#

```
public sealed class FundraisingDbContext :
    DbContext,
    IUnitOfWork
```

```
{
}
```

REPOSITORY / UOW / MAPPING

Application services depend on:

```C#
IUnitOfWork
```

instead of directly referencing:

```C#
FundraisingDbContext
```

29. Is IUnitOfWork Necessary?

Not always.

If your Application already depends on EF Core directly:

```C#
FundraisingDbContext
```

then wrapping:

```C#
SaveChangesAsync()
```

with:

```C#
IUnitOfWork.SaveChangesAsync()
```

may add little.

But an abstraction can be useful when:

EXAMPLE / FLOW

```
Application should not reference EF Core

Several repositories share one commit boundary

Commit semantics need a clear abstraction

Unit tests should control commit behavior
```

This is an architectural trade-off, not a mandatory pattern.

30. Repository Should Not Usually Save Independently

Consider:

```C#
public async Task AddAsync(
    Gift gift)
{
    _dbContext.Gifts.Add(
        gift);
```

```
    await _dbContext.SaveChangesAsync();
}
```

Then:

C#

```
public async Task AddAsync(
    OutboxMessage message)
{
    _dbContext.OutboxMessages.Add(
        message);

    await _dbContext.SaveChangesAsync();
}
```

Application code:

EXAMPLE / FLOW

```
Add Gift
    ↓ commit

Add OutboxMessage
    ↓ commit
```

If the second fails:

EXAMPLE / FLOW

```
Gift exists
Outbox missing
```

The repository has stolen transaction control from the use case.

31. Better Unit-of-Work Boundary

Repository:

C#

```
public async Task AddAsync(
    Gift gift,
    CancellationToken cancellationToken)
{
    await _dbContext.Gifts.AddAsync(
        gift,
        cancellationToken);
}
```

Outbox writer:

C#

```
public async Task AddAsync(
    OutboxMessage message,
    CancellationToken cancellationToken)
{
    await _dbContext.OutboxMessages.AddAsync(
        message,
        cancellationToken);
}
```

Application service:

```C#
await giftRepository.AddAsync(
    gift,
    cancellationToken);

await outboxWriter.AddAsync(
    message,
    cancellationToken);

await unitOfWork.SaveChangesAsync(
    cancellationToken);
```

Now the Application layer controls:

```EXAMPLE / FLOW
business transaction boundary
```

32. SaveChanges Transaction Behavior

By default, when the database provider supports transactions, EF Core applies all changes in a single `SaveChanges` call transactionally. If any change fails, the operation rolls back so the database is not left with a partially applied `SaveChanges`. Microsoft also notes that this default behavior is sufficient for many applications.

Example:

```C#
dbContext.Gifts.Add(
    gift);

dbContext.OutboxMessages.Add(
    outboxMessage);

await dbContext.SaveChangesAsync(
    cancellationToken);
```

Conceptually:

```EXAMPLE / FLOW
BEGIN TRANSACTION

INSERT Gift
INSERT OutboxMessage

COMMIT
```

33. When an Explicit Transaction Is Needed

Suppose one use case needs:

```C#
await dbContext.SaveChangesAsync();

await ExecuteAnotherDatabaseOperationAsync();

await dbContext.SaveChangesAsync();
```

and all of those must succeed/fail together.

Then an explicit transaction may be appropriate:

C#

```
await using IDbContextTransaction transaction =
    await dbContext.Database
        .BeginTransactionAsync(
            cancellationToken);

try
{
    // First set of changes.
    await dbContext.SaveChangesAsync(
        cancellationToken);

    // Additional database work.

    // Second set of changes.
    await dbContext.SaveChangesAsync(
        cancellationToken);

    await transaction.CommitAsync(
        cancellationToken);
}
catch
{
    await transaction.RollbackAsync(
        cancellationToken);

    throw;
}
```

EF Core provides explicit transaction APIs for these scenarios, but Microsoft recommends relying on the default `SaveChanges` transaction when that is sufficient.

34. Keep Transactions Short

Bad:

EXAMPLE / FLOW

```
Open transaction
    ↓
Call payment API
    ↓
Wait 15 seconds
    ↓
Send email
    ↓
Ask user for input
    ↓
Commit SQL
```

Long transactions can hold:

EXAMPLE / FLOW

```
Locks

Database resources

Versioning state
```

and reduce concurrency.

A database transaction should usually contain:

EXAMPLE / FLOW

```
database work
```

REPOSITORY / UOW / MAPPING

not arbitrary long-running remote operations.

35. Database Transaction Does Not Cover External Systems

Suppose:

EXAMPLE / FLOW

```
SQL Server transaction
+
payment HTTP call
+
email provider
```

A normal local EF transaction cannot atomically guarantee all three.

This is exactly why architecture may use:

EXAMPLE / FLOW

```
local database transaction
+
Outbox
+
asynchronous processing
```

instead of trying to hold a SQL transaction open across remote services.

36. Optimistic Concurrency

Unit of Work also intersects with concurrency.

Suppose two requests load:

EXAMPLE / FLOW

```
Gift Version 5
```

Request A saves first.

Request B then tries to save stale state.

With a concurrency token configured, EF Core can detect that the row changed and throw:

C#

```
DbUpdateConcurrencyException
```

rather than silently overwriting the other update.

Microsoft documents EF Core's optimistic concurrency mechanism as comparing the original concurrency token with the current database value during `SaveChanges`.

37. SQL Server rowversion Example

Entity:

C#

```
public sealed class Gift
{
```

```
    public long GiftId
    {
        get;
        private set;
    }

    public byte[] Version
    {
        get;
        private set;
    } = [];
}
```

EF configuration:

```C#
builder
    .Property(
        gift =>
            gift.Version)
    .IsRowVersion();
```

Update SQL conceptually becomes:

```C#
UPDATE fundraising.Gift
SET Status = @status
WHERE GiftId = @giftId
  AND Version = @originalVersion;
```

If zero rows are affected:

```EXAMPLE / FLOW
another transaction probably changed the row
```

and EF can report the concurrency conflict.

PART 3 — SERVICE LAYER

38. What Is the Service Layer Pattern?

Service Layer defines the application's use-case boundary.

It exposes operations such as:

```EXAMPLE / FLOW
Create Gift

Complete Gift

Refund Gift

Assign Campaign

Generate Receipt
```

and coordinates the work required to execute them.

In DDD-oriented architecture, Microsoft describes the Application layer as a thin coordinator: it defines what the software should do, coordinates tasks, and delegates domain r han owning domain state itself.

39. Service Layer Is Not the Same as “Services Folder”

This:

EXAMPLE / FLOW

```
Services/
    GiftService.cs
    CampaignService.cs
    FundService.cs
```

does not automatically mean you have a meaningful Service Layer.

A 2,500-line `GiftService` with:

EXAMPLE / FLOW

```
SQL
HTTP
validation
email
mapping
business rules
logging
transactions
```

is simply a God class placed in a folder named Services.

40. Application Service Example

C#

```
public sealed class CompleteGiftService
{
    private readonly IGiftRepository _giftRepository;
    private readonly IOutboxWriter _outboxWriter;
    private readonly IUnitOfWork _unitOfWork;

    public CompleteGiftService(
        IGiftRepository giftRepository,
        IOutboxWriter outboxWriter,
        IUnitOfWork unitOfWork)
    {
        _giftRepository = giftRepository;

        _outboxWriter = outboxWriter;

        _unitOfWork = unitOfWork;
    }

    public async Task CompleteAsync(
        CompleteGiftCommand command,
        CancellationToken cancellationToken)
    {
        Gift gift =
            await _giftRepository
                .FindForCompletionAsync(
                    command.TenantId,
                    command.GiftId,
                    cancellationToken)
```

```
            ??
            throw new GiftNotFoundException(
                command.GiftId);

        gift.Complete();

        OutboxMessage message =
            OutboxMessage.Create(
                "GiftCompleted",
                new GiftCompletedIntegrationEvent(
                    gift.TenantId,
                    gift.GiftId,
                    gift.TotalAmount));

        await _outboxWriter.AddAsync(
            message,
            cancellationToken);

        await _unitOfWork.SaveChangesAsync(
            cancellationToken);
    }
}
```

The service coordinates:

EXAMPLE / FLOW

```
load aggregate
    ↓
execute domain behavior
    ↓
record integration intent
    ↓
commit
```

41. What Belongs in the Service Layer?

Appropriate responsibilities include:

EXAMPLE / FLOW

```
Use-case orchestration

Calling repositories

Coordinating domain entities

Authorization at use-case level

Selecting integrations

Transaction/commit coordination

Creating application/integration messages

Mapping request data to commands/domain calls

Returning application results
```

42. What Does Not Belong There?

Avoid making Service Layer the home of every business rule.

Bad:

```C#
if (
    gift.Status
    ==
    GiftStatus.Pending
    &&
    gift.Allocations.Sum(
        a => a.Amount)
    ==
    gift.TotalAmount)
{
    gift.Status = GiftStatus.Completed;
}
```

inside the service.

Better:

```C#
gift.Complete();
```

The domain object owns its invariant.

This aligns with Microsoft's guidance that the Application layer coordinates tasks while domain rules remain in the domain model.

43. Application Service vs Domain Service

These are different concepts.

Application Service

Coordinates a use case:

```
EXAMPLE / FLOW
CompleteGiftService
```

May depend on:

```
EXAMPLE / FLOW
Repository
Authorization
UnitOfWork
External integration port
```

Domain Service

Represents business behavior that does not naturally belong to one entity/value object.

Example:

```C#
public sealed class GiftAllocationPolicy
{
    public void EnsureAllocationAllowed(
        Gift gift,
        Fund fund,
        decimal amount)
    {
        // Pure business rules.
    }
}
```

Domain Service should generally not know:

EXAMPLE / FLOW

```
HttpContext
DbContext
SMTP
Controller
```

44. Application Service vs Infrastructure Service

Infrastructure services perform technical work.

Examples:

EXAMPLE / FLOW

```
EmailReceiptSender

AzureBlobDocumentStore

PaymentGatewayAdapter

SqlGiftRepository
```

Application service may call these through interfaces:

EXAMPLE / FLOW

```
Application
    ↓
IReceiptSender
    ↑
EmailReceiptSender
```

45. Service Layer and Controllers

Bad controller:

C#

```
[HttpPost("{giftId:long}/complete")]
public async Task<IActionResult> Complete(
    long giftId)
{
    Gift gift =
        await _dbContext.Gifts
            .Include(...)
            .SingleAsync(...);

    if (...)
    {
        ...
    }

    gift.Status = GiftStatus.Completed;

    await _dbContext.SaveChangesAsync();

    // Publish message.

    return NoContent();
}
```

Better:

```C#
[HttpPost("{giftId:long}/complete")]
public async Task<IActionResult> Complete(
    long giftId,
    CancellationToken cancellationToken)
{
    await _completeGiftService
        .CompleteAsync(
            new CompleteGiftCommand(
                TenantId:
                    _currentTenant.TenantId,
                GiftId:
                    giftId),
            cancellationToken);

    return NoContent();
}
```

Controller owns HTTP.

Service owns the use case.

Domain owns business validity.

46. Service Layer and Vertical Slices

Instead of:

```
GiftService
```

with 35 methods:

```
CreateGift
CompleteGift
RefundGift
CancelGift
ChangeCampaign
AddAllocation
RemoveAllocation
...
```

a vertical-slice design can use:

```
CreateGiftHandler

CompleteGiftHandler

RefundGiftHandler
```

Each one is effectively a focused application service/use-case handler.

This is often easier to maintain than one enormous entity-oriented service class.

47. Transaction Boundary Usually Belongs to the Use Case

The Application layer understands:

EXAMPLE / FLOW

```
which operations together form one business use case
```

REPOSITORY / UOW / MAPPING

Therefore it is often the best place to decide:

EXAMPLE / FLOW

```
when persistence should commit
```

Repositories should focus on persistence access.

Unit of Work performs commit.

Application Service coordinates them.

Diagram 1 — Responsibility Separation

EXAMPLE / FLOW

```
Controller
   |
   | HTTP concerns
   ▼
Application Service
   |
   | use-case orchestration
   ▼
Domain
   |
   | business invariants
   ▼
Repository abstraction
   |
   | persistence operations
   ▼
DbContext / Unit of Work
   |
   | tracking + commit
   ▼
SQL Server
```

PART 4 — MAPPING PATTERNS

48. Why Mapping Exists

Different layers often need different data shapes.

For example, an HTTP create request may look like:

C#

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount,
    IReadOnlyList<GiftAllocationRequest> Allocations);
```

Domain:

C#

```
Gift
```

Database may contain:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Campaign
Fund
```

Response:

C#

```
public sealed record GiftResponse(
    long GiftId,
    decimal Amount,
    string Status,
    string? CampaignName);
```

These types have different purposes.

Mapping is the controlled translation between them.

49. Why Not Use One Model Everywhere?

A common beginner approach is:

EXAMPLE / FLOW

```
Database Entity
    =
Domain Entity
    =
Request DTO
    =
Response DTO
```

This appears simple.

But soon:

EXAMPLE / FLOW

```
Database wants foreign keys.

Domain wants behavior.

API wants client-friendly names.

Response must hide internal properties.

Create request should not contain database-generated ID.

Update request has different validation.

Integration contract must be versioned.
```

One model becomes overloaded with incompatible responsibilities.

50. DTO — Data Transfer Object

A DTO carries data across a boundary.

Examples:

EXAMPLE / FLOW

```
API Request DTO

API Response DTO

Application Result DTO

Integration Message DTO
```

REPOSITORY / UOW / MAPPING

A DTO usually does not contain rich business behavior.

Example:

C#

```
public sealed record GiftSummaryDto(
    long GiftId,
    decimal TotalAmount,
    GiftStatus Status,
    string? CampaignName);
```

Microsoft's Clean Architecture guidance explicitly lists DTOs among types that may belong in the application core when they are needed by application interfaces without introducing UI or Infrastructure dependencies.

51. Request DTO Is Not Necessarily a Domain Entity

Request:

C#

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount);
```

Domain:

C#

```
Gift.Create(
    tenantId,
    request.ConstituentId,
    request.Amount);
```

This allows Domain construction to enforce:

EXAMPLE / FLOW

```
Tenant requirements
Amount rules
Initial status
Other invariants
```

instead of allowing model binding to construct arbitrary domain state.

52. Mapping Boundary

A useful rule is:

> Map at architectural boundaries.

Examples:

EXAMPLE / FLOW

```
HTTP Request
    ↓ map
Application Command
```

EXAMPLE / FLOW

```
Application Result
    ↓ map
HTTP Response
```

EXAMPLE / FLOW

```
Vendor Payment Response
    ↓ map
PaymentResult
```

EXAMPLE / FLOW

```
Database query
    ↓ project
Read DTO
```

Mapping everywhere indiscriminately creates unnecessary code.

53. Manual Mapping

The simplest mapper is ordinary C#.

C#

```
public static GiftResponse ToResponse(
    Gift gift)
{
    return new GiftResponse(
        GiftId:
            gift.GiftId,
        Amount:
            gift.TotalAmount,
        Status:
            gift.Status.ToString(),
        CampaignName:
            gift.Campaign?.Name);
}
```

Advantages:

EXAMPLE / FLOW

```
Explicit

Easy to debug

Compiler checked

Easy to search

No runtime configuration

Business intent visible
```

54. Manual Mapping Is Often Best

For:

EXAMPLE / FLOW

```
5 properties
```

this:

C#

```
return new GiftResponse(
    gift.GiftId,
    gift.TotalAmount,
    gift.Status.ToString(),
    gift.Campaign?.Name);
```

may be clearer than configuring an entire mapping framework.

Do not use AutoMapper merely to avoid writing four assignments.

55. Mapping Extension Methods

You can organize explicit mapping as extension methods:

C#

```
public static class GiftMappings
{
    public static GiftResponse
        ToResponse(
            this Gift gift)
    {
        return new GiftResponse(
            gift.GiftId,
            gift.TotalAmount,
            gift.Status.ToString(),
            gift.Campaign?.Name);
    }
}
```

Usage:

C#

```
GiftResponse response = gift.ToResponse();
```

Good when mapping is:

EXAMPLE / FLOW

```
pure
stateless
simple
```

56. Dedicated Mapper Class

When mapping becomes larger:

C#

```
public sealed class GiftMapper
{
    public GiftResponse ToResponse(
```

```
        Gift gift)
    {
        // Mapping.
    }

    public GiftExportRecord ToExport(
        Gift gift)
    {
        // Mapping.
    }
}
```


This can be useful when mapping involves:

```
EXAMPLE / FLOW
many fields
formatting policies
several related types
testable translation rules
```

But avoid putting database access or major business decisions into a mapper.

57. Mapper Should Not Become Service Layer

Bad:

```
C#
public async Task<GiftResponse> MapAsync(
    Gift gift)
{
    Campaign campaign = await _repository.GetCampaignAsync(...);

    bool authorized = await _authorization.CheckAsync(...);

    // ...
}
```

Now the mapper is:

```
EXAMPLE / FLOW
Repository
Authorization service
Application service
Mapper
```

all at once.

Mapping should generally remain transformation-focused.

58. AutoMapper

AutoMapper is a popular mapping library for .NET that can configure mappings between source and destination types.

Conceptually:

```
C#
CreateMap<Gift, GiftResponse>();
```

then:

```
C#
GiftResponse response =
    mapper.Map<GiftResponse>(
        gift);
```

It can reduce repetitive mapping code, especially when many fields follow predictable conventions.

59. AutoMapper Benefits

AutoMapper can be helpful when:

```
EXAMPLE / FLOW
Large number of DTO mappings

Many properties map conventionally

Flattening is repetitive

Mapping configuration is stable

Team understands the tool
```

Example:

```
C#
public sealed class GiftMappingProfile :
    Profile
{
    public GiftMappingProfile()
    {
        CreateMap<Gift, GiftResponse>()
            .ForMember(
                destination =>
                    destination.Amount,
                options =>
                    options.MapFrom(
                        source =>
                            source.TotalAmount));
    }
}
```

60. AutoMapper Costs

Possible disadvantages:

```
EXAMPLE / FLOW
Mapping behavior less explicit

Runtime configuration

Debugging can be less direct

Complex mappings become difficult to understand

Business logic can accidentally move into mapping configuration

Queryable projections depend on provider translation support
```

A mapping library should reduce accidental boilerplate, not hide important domain behavior.

61. AutoMapper Map vs ProjectTo

This difference is essential.

Map

```C#
Gift entity =
    await dbContext.Gifts
        .Include(...)
        .SingleAsync(...);

GiftResponse dto =
    mapper.Map<GiftResponse>(
        entity);
```

The entity is first loaded into memory.

Then mapping occurs.

ProjectTo

```C#
GiftResponse dto =
    await dbContext.Gifts
        .Where(...)
        .ProjectTo<GiftResponse>(
            mapper.ConfigurationProvider)
        .SingleAsync(
            cancellationToken);
```

`ProjectTo` builds an expression that the query provider can translate.

AutoMapper's official documentation explains that `ProjectTo` emits a projection so an ORM such as EF Core can select the required destination fields rather than first materializing the full source object graph.

62. ProjectTo Must Respect Query Provider Rules

AutoMapper's documentation recommends applying entity-side filtering and sorting before:

```C#
ProjectTo<TDestination>()
```

and placing `ProjectTo` at the end of the LINQ composition so the underlying provider can translate the generated expression correctly.

Good:

```C#
await dbContext.Gifts
    .Where(
        gift =>
            gift.TenantId == tenantId)
    .OrderByDescending(
        gift =>
            gift.GiftDate)
    .ProjectTo<GiftSummaryDto>(
        mapper.ConfigurationProvider)
    .ToListAsync(
        cancellationToken);
```

63. ProjectTo Has Limitations

Not every arbitrary C# mapping operation can be translated into SQL.

AutoMapper documents that query projection supports only mapping operations that can be represented to and understood by the LINQ query provider; many runtime resolvers/converters available to normal `Map` are not supported in query projection.

Therefore:

EXAMPLE / FLOW

```
AutoMapper Map succeeds
```

does not guarantee:

EXAMPLE / FLOW

```
ProjectTo translates successfully.
```

64. Explicit Select vs ProjectTo

Explicit:

C#

```
.Select(
    gift =>
        new GiftSummaryDto(
            gift.GiftId,
            gift.TotalAmount,
            gift.Status,
            gift.Campaign!.Name))
```

Advantages:

EXAMPLE / FLOW

```
Very explicit

Easy to see SQL shape

Compiler-visible

No mapping configuration
```

ProjectTo:

C#

```
.ProjectTo<GiftSummaryDto>(
    mapper.ConfigurationProvider)
```

Advantages:

EXAMPLE / FLOW

```
Reusable projection configuration

Less repetitive mapping

Useful for many DTOs
```

Neither is universally superior.

For critical queries, explicit projections are often easiest to reason about.

65. Mapping Domain to DTO

This is usually safe:

```
EXAMPLE / FLOW
Domain Entity
    ↓
Response DTO
```

because data flows outward.

For example:

```
C#
GiftDetailsDto ToDto(
    Gift gift)
{
    return new(
        gift.GiftId,
        gift.TotalAmount,
        gift.Status);
}
```

66. DTO to Domain Mapping Needs More Care

This is dangerous:

```
C#
mapper.Map(
    updateRequest,
    gift);
```

if `Gift` has important invariants.

Suppose request contains:

```
C#
{
  "status": "Completed",
  "amount": -100
}
```

Blind mapping may bypass:

```
EXAMPLE / FLOW
Gift.Complete()

Amount validation

State-transition rules
```

For rich domain models, prefer explicit behavior:

```
C#
gift.ChangeCampaign(
    request.CampaignId);
```

```
gift.Complete();
```


rather than directly mapping setters.

67. Why ReverseMap Can Be Dangerous

AutoMapper can support reverse mappings, but a technical ability to map both directions does not mean both directions are architecturally safe.

This:

EXAMPLE / FLOW

```
Domain
    →
DTO
```

may be simple data extraction.

But:

EXAMPLE / FLOW

```
DTO
    →
Domain
```

may represent:

EXAMPLE / FLOW

```
business state mutation
```

which should go through domain behavior.

Do not use automatic reverse mapping as a replacement for domain methods.

68. Persistence Model vs Domain Model

There are two major approaches.

Same Entity Model

EXAMPLE / FLOW

```
Domain Gift
    =
EF Core Gift
```

EF maps the domain class directly.

Advantages:

EXAMPLE / FLOW

```
Less duplication
Less mapping
Simpler application
```

Separate Models

```
EXAMPLE / FLOW
Domain Gift

Persistence GiftRow / GiftEntity
```

Repository maps between them.

Advantages:

```
EXAMPLE / FLOW
Maximum persistence independence

Database model can differ heavily

Domain stays completely persistence-specific-free
```

Disadvantages:

```
EXAMPLE / FLOW
Significantly more mapping code

Identity/change tracking complexity

Higher maintenance cost
```

Use separate models only when the architectural value justifies the extra complexity.

69. EF Core Can Map Rich Domain Models

You do not need separate persistence DTOs merely to use a rich domain model.

EF Core can map:

```
EXAMPLE / FLOW
private setters

backing fields

owned/value-object-like structures

relationships

constructor patterns
```

with appropriate configuration.

Therefore:

```
EXAMPLE / FLOW
Domain model != automatically separate persistence model.
```

70. Integration Contract Mapping

Suppose your internal domain event is:

```
C#
public sealed record GiftCompletedDomainEvent(
    Gift Gift);
```

Do not send the entire domain entity over a message broker.

Create an explicit integration contract:


```C#
public sealed record GiftCompletedIntegrationEvent(
    Guid MessageId,
    long TenantId,
    long GiftId,
    decimal Amount,
    DateTimeOffset CompletedAt);
```

Mapping creates a versioned external boundary.

This prevents external consumers from depending on your internal domain representation.

Diagram 2 — Mapping Boundaries

```EXAMPLE / FLOW
HTTP JSON
    ↓
Request DTO
    ↓
Command
    ↓
Domain behavior
    ↓
Persistence
```

Read path:

```EXAMPLE / FLOW
Database
    ↓
Projection
    ↓
Read DTO
    ↓
Response DTO / JSON
```

Integration:

```EXAMPLE / FLOW
Domain state
    ↓
Integration-event mapping
    ↓
Versioned message contract
```

PART 5 — PUTTING THE PATTERNS TOGETHER

71. Create Gift Use Case

Request:

```C#
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal TotalAmount,
    IReadOnlyList<
```

```
        CreateGiftAllocationRequest>
    Allocations);
```

Controller maps request to application command:

```C#
CreateGiftCommand command =
    new(
        TenantId:
            currentTenant.TenantId,
        ConstituentId:
            request.ConstituentId,
        TotalAmount:
            request.TotalAmount,
        Allocations:
            request.Allocations
                .Select(
                    allocation =>
                        new GiftAllocationInput(
                            allocation.FundId,
                            allocation.Amount))
                .ToList());
```

72. Application Service

```C#
public sealed class CreateGiftService
{
    private readonly IGiftRepository _giftRepository;
    private readonly IUnitOfWork _unitOfWork;

    public CreateGiftService(
        IGiftRepository giftRepository,
        IUnitOfWork unitOfWork)
    {
        _giftRepository = giftRepository;

        _unitOfWork = unitOfWork;
    }

    public async Task<CreateGiftResult>
        CreateAsync(
            CreateGiftCommand command,
            CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                command.TenantId,
                command.ConstituentId,
                command.TotalAmount);

        foreach (
            GiftAllocationInput allocation
            in command.Allocations)
        {
            gift.AddAllocation(
                allocation.FundId,
                allocation.Amount);
        }

        await _giftRepository.AddAsync(
            gift,
```

```
            cancellationToken);

        await _unitOfWork.SaveChangesAsync(
            cancellationToken);

        return new CreateGiftResult(
            gift.GiftId,
            gift.TotalAmount,
            gift.Status);
    }
}
```

73. Pattern Responsibilities

Repository:

EXAMPLE / FLOW

```
Add aggregate to persistence context
```

Unit of Work:

EXAMPLE / FLOW

```
Commit changes
```

Service Layer:

EXAMPLE / FLOW

```
Coordinate create-Gift use case
```

Mapping:

EXAMPLE / FLOW

```
Request → Command
Result → Response
```

Domain:

EXAMPLE / FLOW

```
Protect Gift invariants
```

None of these should steal the other's responsibility.

74. Complete Gift + Outbox

C#

```
public async Task CompleteAsync(
    CompleteGiftCommand command,
    CancellationToken cancellationToken)
{
    Gift gift =
        await _giftRepository
            .FindForCompletionAsync(
                command.TenantId,
                command.GiftId,
                cancellationToken)
        ??
        throw new GiftNotFoundException(
            command.GiftId);
```

```
    gift.Complete();

    GiftCompletedIntegrationEvent integrationEvent =
        new(
            MessageId:
                Guid.NewGuid(),
            TenantId:
                gift.TenantId,
            GiftId:
                gift.GiftId,
            Amount:
                gift.TotalAmount,
            CompletedAt:
                _clock.UtcNow);

    OutboxMessage outboxMessage =
        OutboxMessage.Create(
            integrationEvent);

    await _outboxWriter.AddAsync(
        outboxMessage,
        cancellationToken);

    await _unitOfWork.SaveChangesAsync(
        cancellationToken);
}
```

Because Gift and OutboxMessage are saved in the same `SaveChanges`, EF's normal relational transaction behavior can make that commit atomic when supported by the provider.

75. Read-Side Query

C#

```
public Task<GiftDetailsDto?>
    GetGiftAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
{
    return _dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId == tenantId
                &&
                gift.GiftId == giftId)
        .Select(
            gift =>
                new GiftDetailsDto(
                    gift.GiftId,
                    gift.TotalAmount,
                    gift.Status,
                    gift.Campaign == null
                        ? null
                        : gift.Campaign.Name,
                    gift.Allocations
                        .Select(
                            allocation =>
                                new GiftAllocationDto(
                                    allocation.FundId,
                                    allocation.Amount))
                        .ToList()))
        .SingleOrDefaultAsync(
```

```
            cancellationToken);
}
```


This avoids loading an update-oriented aggregate when the use case only needs a read model.

Diagram 3 — Command vs Query Architecture

EXAMPLE / FLOW

```
COMMAND

Controller
    ↓
Application Service
    ↓
Repository
    ↓
Tracked Aggregate
    ↓
Domain Behavior
    ↓
Unit of Work
    ↓
SaveChanges


QUERY

Controller
    ↓
Query Service
    ↓
DbContext
    ↓
AsNoTracking
    ↓
Select / ProjectTo
    ↓
DTO
```

PART 6 — TESTING

76. Repository Testing

A Repository implementation contains:

EXAMPLE / FLOW

```
EF Core query behavior
mapping behavior
database behavior
```

That is infrastructure.

Therefore repository tests are usually:

EXAMPLE / FLOW

```
integration tests
```

rather than pure unit tests.

Microsoft explicitly distinguishes fast unit tests against domain/application logic from integration tests that verify real database behavior.

77. Do Not Mock EF Core Internals Excessively

Testing:

```
C#
DbSet<Gift>
```

with a complex mock often tests your mock more than EF Core.

For repository queries, a real database-compatible integration test is usually more valuable.

For application service tests, fake/mock the repository abstraction if the test goal is:

```
EXAMPLE / FLOW
application orchestration
```

not:

```
EXAMPLE / FLOW
SQL correctness
```

78. Application Service Unit Test

```
C#
[Fact]
public async Task CompleteAsync_CompletesGiftAndCommits()
{
    Gift gift =
        TestGiftFactory
            .FullyAllocatedGift();

    FakeGiftRepository repository = new(gift);

    FakeOutboxWriter outboxWriter = new();

    FakeUnitOfWork unitOfWork = new();

    CompleteGiftService service =
        new(
            repository,
            outboxWriter,
            unitOfWork,
            new FakeClock());

    await service.CompleteAsync(
        new CompleteGiftCommand(
            gift.TenantId,
            gift.GiftId),
        CancellationToken.None);

    Assert.Equal(
        GiftStatus.Completed,
        gift.Status);

    Assert.Single(
        outboxWriter.Messages);

    Assert.Equal(
```

```
        1,
        unitOfWork.SaveCallCount);
}
```

REPOSITORY / UOW / MAPPING

No real SQL is needed because the unit test verifies:

```
EXAMPLE / FLOW
use-case behavior
```

79. Repository Integration Test

```
C#
[Fact]
public async Task FindForCompletionAsync_LoadsAllocations()
{
    await using FundraisingDbContext dbContext = CreateTestDbContext();

    // Arrange persisted test data.

    GiftRepository repository = new(dbContext);

    Gift? gift =
        await repository
            .FindForCompletionAsync(
                tenantId:
                    1,
                giftId:
                    100,
                CancellationToken.None);

    Assert.NotNull(
        gift);

    Assert.NotEmpty(
        gift.Allocations);
}
```

This verifies:

```
EXAMPLE / FLOW
actual EF mapping/query behavior
```

rather than faking it.

PART 7 — PERFORMANCE

80. Avoid Repository N+1 Queries

Bad:

```
C#
foreach (
    Gift gift
    in gifts)
{
    Campaign campaign =
        await campaignRepository.FindAsync(
            gift.CampaignId);
}
```

For 1,000 Gifts:

EXAMPLE / FLOW

```
1 Gift query
+
1,000 Campaign queries
```

Architecture does not excuse inefficient data access.

For read use cases, create the correct database projection/join.

81. Project Only What You Need

For list screens:

C#

```
.Select(
    gift =>
        new GiftListItemDto(
            gift.GiftId,
            gift.TotalAmount,
            gift.Status))
```

is typically preferable to:

C#

```
.Include(...)
.ToListAsync()
```

followed by in-memory mapping when the entities aren't needed for mutation.

EF Core's performance guidance explicitly recommends projecting only required properties.

82. Avoid Premature ToList

Bad:

C#

```
List<Gift> gifts =
    await dbContext.Gifts
        .ToListAsync();

List<GiftSummaryDto> result =
    gifts
        .Where(...)
        .Select(...)
        .ToList();
```

Filtering now occurs in memory.

Better:

C#

```
List<GiftSummaryDto> result =
    await dbContext.Gifts
        .Where(...)
        .Select(...)
        .ToListAsync();
```

Keep query composition database-side until materialization when possible.

83. Repository Method Should Not Materialize Too Early

Bad repository:

```C#
public async Task<List<Gift>>
    GetAllAsync()
{
    return await _dbContext.Gifts
        .ToListAsync();
}
```

Caller wants:

```EXAMPLE / FLOW
only 20 Gifts
for one Tenant
from one date range
```

but repository loads all Gifts.

Repository abstractions must still permit efficient query design.

84. Change Tracker Cost

Tracking is useful for updates, but unnecessary tracking of thousands of read-only entities can consume:

```EXAMPLE / FLOW
memory
CPU
identity tracking structures
```

For read-only reporting:

```C#
AsNoTracking()
```

or direct projection is often appropriate.

85. SaveChanges Batching

One Unit-of-Work commit can be more efficient than repeated individual commits because EF can coordinate multiple changes and batch commands where supported. Microsoft notes that `SaveChanges` can batch multiple modifications and wraps multiple changes transactionally when the provider supports it.

Prefer:

```EXAMPLE / FLOW
Modify several entities
    ↓
SaveChanges once
```

when they belong to the same transaction.

PART 8 — SECURITY

86. Repository Is a Security Boundary Only If Designed as One

This:

```
C#
Task<Gift?> FindAsync(
    long giftId);
```

may be unsafe in a multi-tenant system.

Safer:

```
C#
Task<Gift?> FindAsync(
    long tenantId,
    long giftId,
    CancellationToken cancellationToken);
```

Now the repository contract reminds every caller:

```
EXAMPLE / FLOW
Tenant ownership matters.
```

87. Tenant Filtering

Implementation:

```
C#
return _dbContext.Gifts
    .SingleOrDefaultAsync(
        gift =>
            gift.TenantId == tenantId
            &&
            gift.GiftId == giftId,
        cancellationToken);
```

Do not write:

```
C#
return _dbContext.Gifts
    .FindAsync(giftId);
```

if that allows cross-tenant access.

Repository design should make unsafe access difficult.

88. Mapping Can Leak Sensitive Fields

Suppose entity includes:

```
EXAMPLE / FLOW
PaymentToken
InternalNotes
FraudScore
```

```
TenantId
AuditMetadata
```


Returning the entity directly may expose fields unintentionally.

Explicit response mapping creates an allow-list:

```C#
new GiftResponse(
    gift.GiftId,
    gift.TotalAmount,
    gift.Status);
```

Only intentionally mapped fields leave the system.

89. AutoMapper Security Warning

Convention-based mapping can accidentally include a new property if names align.

For external contracts, review mapping whenever sensitive fields change.

Do not assume:

```EXAMPLE / FLOW
AutoMapper
=
security filter.
```

Security should be explicit.

90. Repository Does Not Replace Authorization

Even if repository enforces:

```EXAMPLE / FLOW
TenantId
```

application authorization may still need:

```EXAMPLE / FLOW
Does current user have permission to refund Gifts?

Can user access this Campaign?

Can this role see donor identity?
```

Tenant filtering and authorization are different concerns.

PART 9 — MEMORY AND THREAD SAFETY

91. Repository Lifetime

Repository using a scoped `DbContext` should normally also be:

```EXAMPLE / FLOW
Scoped
```

Example:

```C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

Microsoft's EF architecture guidance describes sharing one scoped DbContext across repositories participating in the same HTTP-request unit of work.

92. DbContext Is Not Thread-Safe

Do not run:

```C#
Task first = giftRepository.LoadOneAsync();

Task second = giftRepository.LoadAnotherAsync();

await Task.WhenAll(
    first,
    second);
```

if both repository methods share the same `DbContext` and execute concurrently.

EF Core does not support multiple parallel operations on the same context.

93. Long-Lived Unit of Work Is Dangerous

A long-lived DbContext can accumulate:

```EXAMPLE / FLOW
tracked Gift objects
tracked Constituents
navigation graphs
original values
```

causing memory growth and stale tracking state.

EF guidance recommends a fresh DbContext for each unit of work.

PART 10 — COMMON MISTAKES

94. Repository per Table

Bad:

```EXAMPLE / FLOW
GiftAllocationRepository
GiftRepository
CampaignRepository
FundRepository
```

created solely because four tables exist.

Better:

```EXAMPLE / FLOW
repository boundaries based on aggregate/use-case needs
```

95. Repository Calls SaveChanges

Bad:

```C#
public async Task AddAsync(
    Gift gift)
{
    _dbContext.Gifts.Add(
        gift);

    await _dbContext.SaveChangesAsync();
}
```

This prevents a broader transaction containing:

EXAMPLE / FLOW

```
Gift
+
OutboxMessage
```

unless transaction control is moved elsewhere.

96. Service Layer Contains Domain Logic

Bad:

```C#
if (...)
{
    gift.Status = GiftStatus.Completed;
}
```

Better:

```C#
gift.Complete();
```

97. Domain Entity Contains Repository

Bad:

```C#
public class Gift
{
    public async Task CompleteAsync(
        IGiftRepository repository)
    {
    }
}
```

Domain objects should generally not orchestrate persistence.

98. Repository Returns DTOs for Everything

Repository is usually a persistence boundary for domain/application persistence concerns.

If every repository method returns:

EXAMPLE / FLOW

```
GiftDto
CampaignDto
FundDto
```

REPOSITORY / UOW / MAPPING

you may actually be building:

EXAMPLE / FLOW

```
query service
```

rather than a repository.

That can be fine—just name and design it according to its real responsibility.

99. Mapping Business Rules

Bad mapping:

C#

```
.ForMember(
    destination =>
        destination.Status,
    options =>
        options.MapFrom(
            source =>
                source.Amount > 10000
                    ? "Major"
                    : "Normal"));
```

if:

EXAMPLE / FLOW

```
Major Gift
```

is an important business classification.

Put business policy in the appropriate domain/application rule.

Mapping should consume that policy result rather than secretly define it.

100. Blind Reverse Mapping

Bad:

C#

```
mapper.Map(
    request,
    gift);
```

when the request can change:

EXAMPLE / FLOW

```
Amount
Status
Tenant
Allocations
```

This can bypass invariants.

Use explicit domain methods.

101. Generic Repository Hiding EF Features

Bad:

```
EXAMPLE / FLOW
Generic repository exposes
only GetById/GetAll/Add
```

but application needs:

```
EXAMPLE / FLOW
efficient projection
pagination
aggregate-specific Includes
ExecuteUpdate
temporal queries
```

Developers eventually work around the repository.

An abstraction that constantly needs bypassing is probably the wrong abstraction.

102. Mocking Repository Without Integration Tests

A fake repository proves:

```
EXAMPLE / FLOW
Application service coordinates correctly.
```

It does not prove:

```
EXAMPLE / FLOW
EF query works.

SQL mapping works.

Constraint works.

Include is correct.

Transaction works.
```

Both unit and integration tests have roles.

103. Mapping After Loading Huge Graph

Bad:

```
C#
List<Gift> gifts =
    await dbContext.Gifts
        .Include(g => g.Campaign)
        .Include(g => g.Allocations)
        .ThenInclude(a => a.Fund)
        .ToListAsync();

return mapper.Map<
    List<GiftSummaryDto>>(
        gifts);
```

for a screen requiring:

EXAMPLE / FLOW

```
GiftId
Amount
CampaignName
```

Use database projection instead.

PART 11 — BEST PRACTICES

104. Repository Best Practices

Use custom repositories when they provide meaningful persistence boundaries.

Prefer repository methods that represent aggregate/use-case needs:

C#

```
FindForCompletionAsync(...)
```

rather than generic persistence plumbing everywhere.

Keep EF Core implementation details inside Infrastructure if persistence independence matters.

Do not create repositories automatically for every table.

Do not force read-only reporting queries through aggregate repositories when a direct query projection is clearer.

105. Unit of Work Best Practices

Treat one `DbContext` as one short unit of work.

Share one scoped context among repositories participating in the same use case.

Commit once where possible.

Let the Application layer define transaction boundaries.

Use explicit transactions only when multiple persistence steps genuinely require one transaction beyond the normal `SaveChanges` boundary.

Keep remote I/O outside long-running database transactions.

106. Service Layer Best Practices

Keep application services focused on one use case.

Prefer:

EXAMPLE / FLOW

```
CreateGiftService
CompleteGiftService
RefundGiftService
```

over one massive:

EXAMPLE / FLOW

```
GiftService
```

when the feature set grows.

Application services should orchestrate.

Domain objects should enforce invariants.

Infrastructure should implement technical details.

Controllers should handle HTTP.

107. Mapping Best Practices

Map at boundaries.

Prefer explicit mapping for important domain/API transformations.

Use AutoMapper where it genuinely reduces repetitive mapping.

Use direct `Select` or `ProjectTo` for database-side read projections.

Do not automatically map update DTOs directly onto rich domain entities.

Do not put authorization, SQL access, or major business decisions inside mappers.

Keep public API and integration contracts explicit.

PART 12 — PRACTICAL EXERCISES

Exercise 1 — Repository Boundary

Given:

EXAMPLE / FLOW

```
Gift
GiftAllocation
```

with Gift owning allocations, design:

C#

```
IGiftRepository
```

without creating:

C#

```
IGiftAllocationRepository
```

for ordinary allocation updates.

Exercise 2 — Remove SaveChanges from Repository

Refactor:

C#

```
public async Task AddAsync(
    Gift gift)
{
    _dbContext.Gifts.Add(
        gift);

    await _dbContext.SaveChangesAsync();
}
```

to:

```C#
public void Add(
    Gift gift)
{
    _dbContext.Gifts.Add(
        gift);
}
```

and move commit responsibility to the Application service.

Exercise 3 — Add Outbox Transaction

Create:

```EXAMPLE / FLOW
Gift status update
+
OutboxMessage insert
```

then perform one:

```C#
SaveChangesAsync()
```

so they share the same EF transaction.

Exercise 4 — Read Projection

Replace:

```C#
await dbContext.Gifts
    .Include(g => g.Campaign)
    .ToListAsync();
```

plus in-memory mapping with:

```C#
.Select(
    gift =>
        new GiftSummaryDto(
            gift.GiftId,
            gift.TotalAmount,
            gift.Campaign!.Name))
```

Exercise 5 — Service Layer

Move:

```EXAMPLE / FLOW
Load Gift
Complete Gift
Create OutboxMessage
Commit
```

from the controller into:

```EXAMPLE / FLOW
CompleteGiftService
```

Exercise 6 — Domain Rule

Move this out of Application service:

```
C#
if (
    allocations.Sum(
        x => x.Amount)
    !=
    gift.TotalAmount)
{
    throw ...
}
```

into:

```
C#
gift.Complete();
```

Exercise 7 — Manual Mapping

Create:

```
C#
GiftResponse ToResponse(
    GiftDetailsDto result);
```

without AutoMapper.

Compare readability with an AutoMapper profile.

Exercise 8 — AutoMapper Projection

Configure:

```
EXAMPLE / FLOW
Gift
    →
GiftSummaryDto
```

then compare:

```
C#
ProjectTo<GiftSummaryDto>()
```

against explicit:

```
C#
Select(...)
```

Inspect generated SQL.

Exercise 9 — Multi-Tenant Repository

Change:

```
C#
FindAsync(
    giftId)
```

to:

```
C#
FindAsync(
    tenantId,
    giftId,
    cancellationToken)
```

and ensure every query filters by tenant.

Exercise 10 — Concurrency

Add:

```
C#
byte[] Version
```

as a SQL Server rowversion.

Simulate two requests modifying the same Gift and handle:

```
C#
DbUpdateConcurrencyException
```

Exercise 11 — Repository Integration Test

Test:

```
C#
FindForCompletionAsync()
```

against a real relational test database and verify allocations are loaded.

Exercise 12 — Architecture Review

Take each data-access class and ask:

```
EXAMPLE / FLOW
Is this really a Repository?

Is it actually a Query Service?

Is it just DbSet renamed?

Does it represent an aggregate boundary?

Does it preserve efficient querying?

Who controls SaveChanges?
```

PART 13 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is the Repository pattern?

Repository is a persistence abstraction that mediates between application/domain logic and data-access implementation.

Instead of application code containing EF Core query details everywhere, it can request domain-oriented operations such as:

```
C#
FindForCompletionAsync(...)
```

The repository implementation translates those requests into EF Core/database operations.

Microsoft describes Repository as an intermediary between the domain recommends aggregate-oriented repositories for DDD-style systems.

2. Does EF Core require the Repository pattern?

No.

EF Core's `DbContext` and `DbSet<T>` already implement many Repository and Unit-of-Work behaviors.

Microsoft explicitly states that custom repositories are optional and direct `DbContext` usage can be appropriate when it produces the simplest design.

3. When should you create a custom Repository over DbContext?

A custom Repository is useful when it creates a meaningful boundary, for example:

EXAMPLE / FLOW

```
Aggregate-specific loading

Domain-oriented persistence operations

Persistence isolation

Reusable query/update semantics

Test seam for application orchestration
```

If it simply renames `DbSet<T>` methods, its value is questionable.

4. Should there be one Repository per database table?

Usually no in a DDD-style architecture.

Repositories should normally align with aggregate roots rather than every table.

For example:

EXAMPLE / FLOW

```
Gift
    └─ GiftAllocation
```

may use one `IGiftRepository` because Gift controls allocation consistency.

Microsoft's DDD persistence guidance specifically recommends one repository per aggregate root rather than one per database table.

5. What are the advantages and disadvantages of a generic Repository?

Advantages include reducing repeated CRUD infrastructure.

Disadvantages include forcing unrelated aggregates into the same abstraction, hiding useful EF Core capabilities, creating lowest-common-denominator APIs, and encouraging repository-per-entity design.

Use generics only where persistence behavior is genuinely generic.

6. Should a Repository return `IQueryable<T>`?

It can, but doing so weakens the abstraction.

`IQueryable<T>` lets callers build efficient provider-translated queries, but it also exposes query-provider semantics, deferred execution, tracking decisions, and EF-style composition.

If the purpose of Repository is persistence isolation, exposing `IQueryable<T>` may undermine that purpose.

For read-heavy CQRS designs, a separate query service is often clearer.

7. What is the Unit of Work pattern?

Unit of Work coordinates multiple persistence changes as one logical operation and provides a commit boundary.

For example:

EXAMPLE / FLOW

```
Update Gift
Insert OutboxMessage
    ↓
SaveChanges
```

should be one unit if those changes must succeed together.

8. How does DbContext implement Unit of Work?

`DbContext` tracks loaded/attached entities and their modifications through its change tracker.

When `SaveChanges` executes, EF translates the accumulated state changes into database operations.

Microsoft explicitly describes `DbContext` as a short-lived Unit-of-Work implementation.

9. Do you need an IUnitOfWork abstraction on top of DbContext?

Not always.

It can be useful when the Application layer should not reference EF Core or when multiple repositories need a clear common commit abstraction.

But if Application already intentionally uses `DbContext`, wrapping only `SaveChangesAsync` may simply add another layer.

The decision should be architectural, not ceremonial.

10. Should Repository methods call SaveChanges internally?

Usually not when several repositories/operations may participate in one business transaction.

If each repository commits independently:

EXAMPLE / FLOW

```
Gift saved
    ↓
Outbox save fails
```

you may leave partial state.

A better design often lets the Application/Unit-of-Work boundary decide when to commit.

11. Is SaveChanges transactional?

For providers that support transactions, EF Core normally applies all changes in one `SaveChanges` call inside a transaction so either all changes succeed or the operation rolls back.

Microsoft says this default transaction behavior is sufficient for many applications.

12. When do you need an explicit EF Core transaction?

Use an explicit transaction when one atomic business operation spans multiple `SaveChanges` calls or additional database operations that must commit together.

Do not open an explicit transaction automatically around every use case; the normal `SaveChanges` transaction may already provide the required atomicity.

13. What is the Service Layer pattern?

Service Layer defines application use cases and coordinates work across Domain, repositories, transactions, and external capabilities.

For example:

EXAMPLE / FLOW

```
CompleteGiftService
```

might:

EXAMPLE / FLOW

```
load Gift
invoke Gift.Complete()
write OutboxMessage
commit
```

It should coordinate domain behavior rather than reproduce the business rules itself.

14. What is the difference between Application Service and Domain Service?

An **Application Service** coordinates a use case and may interact with repositories, authorization, transactions, or integrations.

A **Domain Service** contains domain/business behavior that does not naturally belong to one entity/value object.

Domain Service should remain focused on business policy rather than infrastructure.

15. What is the difference between Service Layer and a Repository?

Service Layer answers:

EXAMPLE / FLOW

```
What steps make up this application use case?
```

Repository answers:

EXAMPLE / FLOW

```
How does application/domain code obtain or persist an aggregate?
```

Service Layer may use one or more repositories.

They solve different problems.

16. Why should business rules not live primarily in Application Services?

If every rule is implemented in services:

EXAMPLE / FLOW

```
Gift becomes an anemic bag of properties
```

and different use cases can bypass rules.

For invariants such as:

```
EXAMPLE / FLOW
Gift allocations must equal Gift amount before completion
```

the Gift domain object is usually the stronger boundary.

The Application layer should call:

```
C#
gift.Complete();
```

rather than duplicate completion rules.

17. What is a DTO?

A DTO, or Data Transfer Object, is a data shape designed to move information across a boundary.

Examples include:

```
EXAMPLE / FLOW
API request

API response

query result

integration message
```

A DTO is not automatically a domain entity and normally contains little or no domain behavior.

18. Why should API DTOs be separate from domain entities?

API contracts and domain models serve different purposes.

The API contract needs:

```
EXAMPLE / FLOW
stable external shape
security filtering
client-friendly structure
versioning
```

while the domain entity needs:

```
EXAMPLE / FLOW
business state
invariants
behavior
```

Separate types prevent API evolution from dictating domain design and reduce accidental exposure of internal fields.

19. What are the benefits of manual mapping?

Manual mapping is:

```
EXAMPLE / FLOW
explicit
compiler visible
```

```
easy to debug
easy to review
```

For small or important mappings, direct C# is often clearer than a mapping framework.

Example:

```C#
new GiftResponse(
    gift.GiftId,
    gift.TotalAmount,
    gift.Status);
```

20. When is AutoMapper useful?

AutoMapper is most useful when an application contains many repetitive mappings with conventional property relationships.

It can reduce boilerplate and centralize mapping configuration.

It becomes less attractive when mappings contain complex business logic or when developers can no longer easily understand what data is being transformed.

21. What is the difference between AutoMapper Map and ProjectTo?

`Map` maps objects already materialized in memory.

`ProjectTo` builds a LINQ expression for an `IQueryable`, allowing an ORM such as EF Core to translate the projection into SQL.

AutoMapper recommends applying filters/sorting before `ProjectTo` and using it as the final LINQ projection step.

22. Why can direct Select be better than mapping entities after loading?

Direct projection:

```C#
.Select(
    gift =>
        new GiftSummaryDto(...))
```

lets EF request only the columns required by the result.

Loading complete entities first may retrieve unnecessary database columns and create unnecessary tracking/materialization work.

EF Core's performance guidance recommends projecting only needed properties.

23. Why is blindly mapping DTOs back into domain entities dangerous?

A rich domain entity may require behavior such as:

```C#
gift.Complete();
gift.ChangeCampaign(...);
gift.AddAllocation(...);
```

to protect invariants.

Blindly mapping request properties directly onto entity setters can bypass these rules.

Automatic mapping is safest for data transformation, not as a substitute for domain behavior.

24. How should command and query persistence differ in a CQRS-style application?

Commands often benefit from:

```
EXAMPLE / FLOW
Repository
    ↓
Tracked aggregate
    ↓
Domain behavior
    ↓
Unit of Work
```

Queries can often use:

```
EXAMPLE / FLOW
DbContext
    ↓
AsNoTracking
    ↓
Select / ProjectTo
    ↓
DTO
```

without reconstructing rich aggregates.

Microsoft's DDD persistence guidance explicitly allows CQRS queries to use separate query paths rather than requiring all reads to go through aggregate repositories.

25. How would you use Repository, Unit of Work, Service Layer, and Mapping in the Fundraising Platform?

A practical write flow is:

```
EXAMPLE / FLOW
POST /gifts/{id}/complete
        ↓
CompleteGiftRequest
        ↓
CompleteGiftCommand
        ↓
CompleteGiftService
        ↓
IGiftRepository
        ↓
Tracked Gift aggregate
        ↓
Gift.Complete()
        ↓
OutboxMessage
        ↓
IUnitOfWork.SaveChangesAsync()
```

The responsibilities are:

```
EXAMPLE / FLOW
Repository
    →
Load/persist Gift aggregate

Unit of Work
```

```
    →
Commit Gift + Outbox atomically

Service Layer
    →
Coordinate completion use case

Domain
    →
Enforce completion rules

Mapping
    →
Translate HTTP/application/integration shapes
```

For reads:

EXAMPLE / FLOW

```
GET /gifts/{id}
        ↓
GiftQueries
        ↓
DbContext.AsNoTracking()
        ↓
Select / ProjectTo
        ↓
GiftDetailsDto
        ↓
GiftResponse
```

This keeps write-side business consistency strong while allowing read queries to remain efficient.

PART 14 — QUICK REVISION

Concept	Meaning
Repository	Persistence abstraction around aggregate/application needs
Aggregate Repository	Usually one repository per aggregate root
Generic Repository	Shared generic persistence abstraction
`IQueryable` Repository	Flexible but leaks provider/query semantics
DbContext	Already provides Repository/UoW-like behavior
Unit of Work	Coordinates change set + commit
Change Tracker	Tracks entity state changes
`SaveChanges`	Persists tracked changes
Default transaction	One `SaveChanges` is transactional when provider supports it
Application Service	Coordinates use case
Domain Service	Business behavior not naturally owned by one entity
Infrastructure Service	Technical implementation
DTO	Boundary-oriented data shape
Manual Mapping	Explicit C# transformation

Concept	Meaning
AutoMapper	Convention/configuration-based mapper
`Map`	In-memory object mapping
`ProjectTo`	IQueryable/database-side projection
`Select` projection	Explicit query DTO projection
Tracking	Useful when entity will be modified
`AsNoTracking`	Useful for read-only queries
Optimistic concurrency	Detect stale writes at save
rowversion	SQL Server concurrency token option
Command path	Repository + domain + UoW
Query path	Projection directly to DTO
Outbox	Commit state and integration intent together

PART 15 — FINAL DECISION FLOW

EXAMPLE / FLOW

```
Need to update domain state?
        ↓
Yes
        ↓
Is there meaningful domain behavior?
        ↓
Yes
        ↓
Load aggregate
        ↓
Repository
        ↓
Domain method
        ↓
Unit of Work
        ↓
SaveChanges
```

For simple CRUD:

EXAMPLE / FLOW

```
No meaningful domain boundary?
        ↓
Direct DbContext may be simpler
```

For a read query:

EXAMPLE / FLOW

```
Need to modify entity?
        ↓
No
        ↓
DbContext
        ↓
AsNoTracking
        ↓
```

```
Where / OrderBy
      ↓
Select or ProjectTo
      ↓
DTO
```

For mapping:

EXAMPLE / FLOW

```
Simple important transformation?
      ↓
Manual mapping
```

EXAMPLE / FLOW

```
Many repetitive conventional mappings?
      ↓
Consider AutoMapper
```

EXAMPLE / FLOW

```
Database query projection?
      ↓
Select
or
ProjectTo
```

EXAMPLE / FLOW

```
DTO → rich domain update?
      ↓
Prefer explicit domain methods
rather than blind mapping
```

PART 16 — FINAL END-TO-END FUNDRAISING FLOW

EXAMPLE / FLOW

```
CLIENT
  ↓
POST /api/gifts
  ↓
CreateGiftRequest
  ↓
Controller
  ↓ map
CreateGiftCommand
  ↓
CreateGiftService
  ↓
Gift.Create()
  ↓
Gift.AddAllocation()
  ↓
IGiftRepository.AddAsync()
  ↓
IUnitOfWork.SaveChangesAsync()
  ↓
EF Core DbContext
  ↓
SQL Server
```

Completion flow:

EXAMPLE / FLOW

REPOSITORY / UOW / MAPPING

```
CLIENT
   ↓
POST /api/gifts/100/complete
   ↓
CompleteGiftCommand
   ↓
CompleteGiftService
   ↓
IGiftRepository.FindForCompletionAsync()
   ↓
Tracked Gift Aggregate
   ↓
gift.Complete()
   ↓
Create GiftCompletedIntegrationEvent
   ↓
Create OutboxMessage
   ↓
SaveChanges once
   ↓
Gift + OutboxMessage committed together
```

Read flow:

EXAMPLE / FLOW

```
CLIENT
   ↓
GET /api/gifts/100
   ↓
Gift Query Service
   ↓
DbContext.Gifts
   ↓
AsNoTracking
   ↓
Where TenantId + GiftId
   ↓
Select / ProjectTo
   ↓
GiftDetailsDto
   ↓
Response mapping
   ↓
JSON
```

The core architectural rule is:

> **Repository, Unit of Work, Service Layer, and Mapping are useful only when each one protects a real boundary. Repository should express meaningful persistence access rather than merely rename `DbSet`; Unit of Work should define a real commit boundary rather than wrap `SaveChanges` ceremonially; Service Layer should coordinate use cases rather than absorb all domain logic; and mapping should translate between boundaries without becoming a hidden place for business rules.**

For the Fundraising Platform, a strong practical approach is:

EXAMPLE / FLOW

```
WRITE SIDE
```

```
Application Service
    ↓
Repository
    ↓
Tracked Domain Aggregate
    ↓
Domain Methods
    ↓
Unit of Work
    ↓
SaveChanges
```

REPOSITORY / UOW / MAPPING

and:

EXAMPLE / FLOW

```
READ SIDE

Query Service
    ↓
EF Core IQueryable
    ↓
AsNoTracking
    ↓
Select / ProjectTo
    ↓
DTO
```

This gives you:

EXAMPLE / FLOW

```
strong business invariants
clear persistence ownership
efficient queries
explicit transaction boundaries
clean API contracts
testable use cases
```

without creating unnecessary abstraction around every EF Core operation.

Next roadmap topic: Topic 36 — Unit Testing and Testable C#.